

Weather Crisis Project Report

Maftunakhon Makhkamova and Annalin Dequental

Project Description:

This project presents a weather crisis emergency system that simulates how individuals affected by disasters request assistance and are assigned to appropriate shelters. The system allows users to perform key operations such as adding shelters, submitting help requests, viewing shelter information and status, assigning shelters based on availability and proximity, and reviewing completed assignments. By implementing these features, the project demonstrates how programming concepts can be applied to model real-world emergency response scenarios and support decision-making processes in crisis situations.

Significance of the Project:

In the creation of this project, it was important to consider not only what programming techniques and concepts we would utilize, but also how they could correlate to aspects of a weather crisis application. So, in designing a weather emergency system, based upon our knowledge of data structures and the principles of Object-Oriented Programming from this course, it would be best to organize our program through various classes. The program comprises three classes, HelpRequest, Shelter, and EmergencySystem, where HelpRequest and Shelter objects act as variables to be stored and tracked by the EmergencySystem as a whole. This makes the project meaningful as it models how resources can be managed and allocated efficiently during emergency situations.

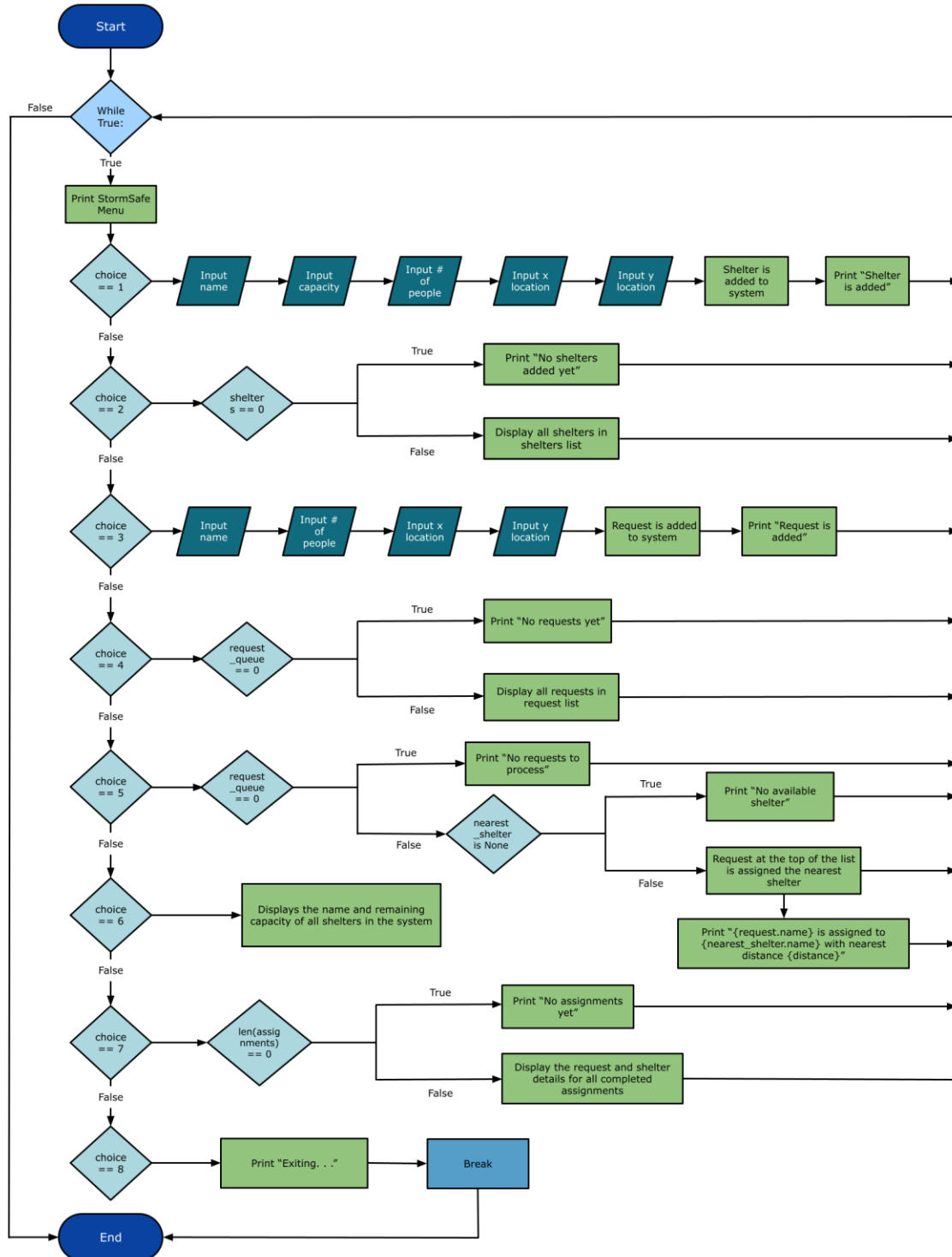
Installation and Usage Instructions:

The program requires Python 3 to run. All project files (`main.py`, `shelter.py`, `request.py`, and `emergency_system.py`) should be placed in the same directory. To execute the program, run the `main.py` file using a Python interpreter: `python main.py`. Once the program starts, a menu will be displayed in the terminal. The user can interact with the system by entering the number corresponding to the desired action. The system will then prompt for the necessary inputs, such as shelter details or help request information. The program continues running until the user selects the exit option.

Code Structure:

Our program is organized into the classes of HelpRequest, EmergencySystem, and Shelter. The overall functionality of these classes and their methods is run through the `main.py` module. In the module, the user interactions between the emergency system application and user are controlled by a while loop which outlines the eight possible user actions: add shelter, view shelters, add help request, view requests, assign shelter, view shelter status, view assignments, and exit.

Flow Chart:



Functionalities:

EmergencySystem Class:

- **add_shelter:** adds a shelter to the list of shelters saved in the system
- **add_request:** adds a request to the list of help requests made to the system
- **view_shelters:** displays all the shelters currently saved in the system's shelter list
- **view_requests:** displays all the help requests saved to the system's request list
- **view_assignments:** displays all assignment that have been completed including their details such as distance and shelter names
- **find_nearest_shelter:** finds the closest shelter to a help request based upon its location, capacity, and calculated distance from other shelters
- **assign_shelter:** assigns a help request a shelter based upon the nearest shelter
- **view_shelter_status:** displays the remaining capacity for assignment to each shelter

HelpRequest Class

- **show_request:** displays the details of a given help request

Shelter Class

- **has_space:** checks if a specific shelter has available space for more occupants or not
- **add_people:** adds people to a specific shelter
- **remaining_capacity:** calculates the amount of people able to be assigned to a shelter before it becomes full capacity
- **show_shelter:** displays a shelter's name, location, and occupant count

Main.py

- User input = 1: user can add a shelter to the system
- User input = 2: user can view shelters within the system
- User input = 3: user can submit a help request
- User input = 4: user can view requests
- User input = 5: user can assign shelters from the request queue
- User input = 6: user can view the status of shelters
- User input = 7: user can view shelter assignments
- User input = 8: user can exit the weather crisis emergency response system

Testing Results and Verification of Function:

- The addition, storage, and display of shelters function correctly when tested with sample inputs.

```
--- StormSafe Menu ---
1. Add Shelter
2. View Shelters
3. Add Help Request
4. View Requests
5. Assign Shelter
6. View Shelter Status
7. View assignments
8. Exit
Enter choice: 1
Shelter name: Abington
Capacity: 200
Current people: 46
X location: 60
Y location: 75
Shelter is added
```

```
--- StormSafe Menu ---
1. Add Shelter
2. View Shelters
3. Add Help Request
4. View Requests
5. Assign Shelter
6. View Shelter Status
7. View assignments
8. Exit
Enter choice: 2
Shelter name: Abington
Shelter coordinates: (60, 75)
Shelter capacity: 200
Shelter current people count: 46
-----
```

```
--- StormSafe Menu ---
1. Add Shelter
2. View Shelters
3. Add Help Request
4. View Requests
5. Assign Shelter
6. View Shelter Status
7. View assignments
8. Exit
Enter choice: 6
Abington has 154 space
Harrisburg has 125 space
```

- Requests and assignments are stored and displayed accurately.
- The system correctly assigns shelters based on distance and capacity.
- The program handles both valid and invalid inputs appropriately.
- The exit functionality terminates the program as expected.

```
--- StormSafe Menu ---
1. Add Shelter
2. View Shelters
3. Add Help Request
4. View Requests
5. Assign Shelter
6. View Shelter Status
7. View assignments
8. Exit
Enter choice: 3
Request name: Lucia
Number of people: 50
X location: 70
Y location: 23
Request is added
```

```
--- StormSafe Menu ---
1. Add Shelter
2. View Shelters
3. Add Help Request
4. View Requests
5. Assign Shelter
6. View Shelter Status
7. View assignments
8. Exit
Enter choice: 4
Name: Lucia
People: 50
Location: (70, 23)
-----
```

```
--- StormSafe Menu ---
1. Add Shelter
2. View Shelters
3. Add Help Request
4. View Requests
5. Assign Shelter
6. View Shelter Status
7. View assignments
8. Exit
Enter choice: 5
50 people added to Harrisburg
Lucia is assigned to Harrisburg with nearest distance 23.32
```

```

--- StormSafe Menu ---
1. Add Shelter
2. View Shelters
3. Add Help Request
4. View Requests
5. Assign Shelter
6. View Shelter Status
7. View assignments
8. Exit
Enter choice: 7
Request name: Anna
People assigned: 25
Assigned shelter: Abington
Shelter location: (60, 75)
Distance: 39.85
-----
Request name: Lucia
People assigned: 50
Assigned shelter: Harrisburg
Shelter location: (50, 35)
Distance: 23.32
-----

```

```

--- StormSafe Menu ---
1. Add Shelter
2. View Shelters
3. Add Help Request
4. View Requests
5. Assign Shelter
6. View Shelter Status
7. View assignments
8. Exit
Enter choice: a
Invalid choice

```

```

--- StormSafe Menu ---
1. Add Shelter
2. View Shelters
3. Add Help Request
4. View Requests
5. Assign Shelter
6. View Shelter Status
7. View assignments
8. Exit
Enter choice: 8
Exiting...

```

Achievement of Project Goals:

The project successfully achieves its goal of modeling a simplified weather crisis emergency system through a command-line interface. The execution results demonstrate that the system correctly handles the core processes of adding shelters, storing help requests, and assigning individuals to the nearest available shelter based on capacity and distance.

The captured outputs confirm that shelters are dynamically updated as people are assigned, and that requests are processed in a first-in, first-out manner. The system also accurately calculates distances using location coordinates and ensures that assignments only occur when sufficient capacity is available.

AI Usage:

In designing this system, AI was used as an aid for finding the overall function for our application, organizing our code, and dividing the work within our group. For instance, ChatGPT helped us explore suitable project directions based on course concepts and refine the structure of our application. Additionally, AI was used to clarify programming concepts and improve the organization of the code. It also supported the division of work between team members, where one focused on shelters and the other on request management, while both collaborated on the overall system.

Issues, Limitations, and Application of Course Learning:

During the development of this project, several challenges and limitations were identified. One issue encountered was ensuring that shelters were correctly assigned based on both capacity and distance. The system also has functional limitations. It operates entirely through a command-line interface, which restricts visualization and user interaction compared to graphical applications. The model assumes a simplified environment using coordinate-based locations and does not

account for real-world constraints such as road conditions, traffic, or dynamic changes during a crisis. Furthermore, requests are processed in a basic first-in, first-out manner without advanced prioritization based on urgency or severity.

Despite these limitations, the project effectively demonstrates the application of key concepts learned in the course. Object-Oriented Programming was used to structure the system into logical classes (HelpRequest, Shelter, and EmergencySystem), promoting modularity and organization. Data structures such as lists were used to store shelters and manage the request queue, which follows a First-In, First-Out (FIFO) approach to ensure that requests are processed in the order they are received. Additionally, algorithmic thinking was applied in implementing the nearest-shelter selection using distance calculations. Control flow structures, including loops and conditionals, enabled the menu-driven interface and decision-making processes.

Overall, the project highlights how foundational programming concepts can be applied to design a functional system that reflects aspects of real-world problem solving, while also identifying areas for future improvement and expansion.

Overall Conclusions:

The project demonstrates a well-developed and functional emergency response system that successfully applies key concepts from the course. The code is organized using object-oriented design, and core features such as shelter management, request processing, and FIFO-based assignments are implemented effectively. The report is clear, structured, and supported by relevant examples. Overall, the project meets its objectives and reflects a solid understanding of both programming and real-world problem modeling.